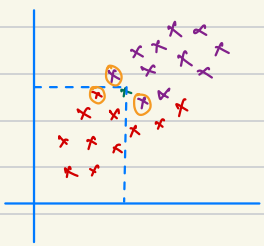
K Nearest Neighbours

	cgpa	iq	placement
100	8	80	1
	7	70	0
	-	-	-
	-	-	-

→ Model → cgpa → iq → } Predicted.



x → 0

x → 1

x → Query point.

k = 3

↑

3 nearest neighbours.

Finding all euclidean distances for 'x' Query Point

Let's say 100 distances.

↓

Sort in asc. order.

↓

Closest point comes in starting.

↓

Applying majority count (there, 1 in majority)

↓

falls under class '1'.

KNN can work for n-dimensions.

x_1	x_2	x_3	x_4	x_5	..... x_n
-	-	-	-	-	- - - - - -
-	-	-	-	-	- — - - - - —

→ 3 vectors → majority count.

Accuracy score. = $\frac{\text{Total Correct Prediction}}{\text{Total.}}$

How to select 'k' ?

Data.

Heuristic Approach.

$k = \sqrt{n}$

Avoid even value for k.

Experimentation.

(Better technique)

Cross-validation.

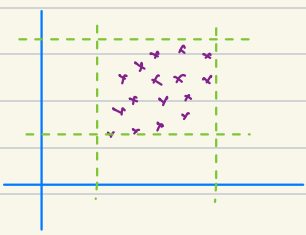
n = 1000

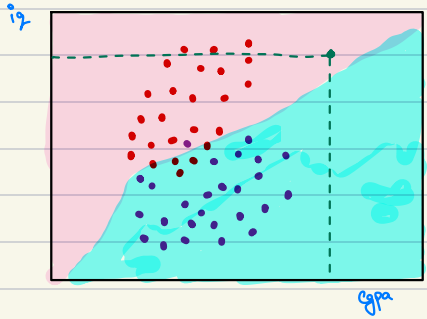
test = 800 train = 200.

Decision Surface :

Decision Surface → It's a tool for classification algorithm. (like kNN, svm)

cgpa | iq | pass [0, 1]





Step 1: Plot training data.

2: On X, Y axis, calculate range.

3: Generate numpy meshgrid for the range.

4: On training data → train kNN model.

5: Every generated point of meshgrid is given to kNN model [0, 1] → [Red, Blue] encoding is performed.

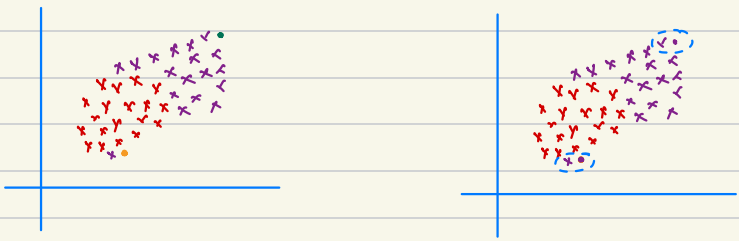
6: Every point is shown as pixel.

7: Decision surface is created.

Use mlxtend to create decision surface.

Overfitting & Underfitting in kNN :

When k=1



It's an example of overfitting.

When k = n (200)

It's an example of underfitting.

Limitations of KNN :

1. for large dataset. → n_col = 100, n_rows = 500000 (500000, 100)
   - ↳ KNN is a lazy learning technique.
     - ↳ Everything is done under predict phase, not during training phase.
2. High dimensional data. → n_cols = 500 features.
   - ↳ Curse of dimensionality. → Reliability of distance is weak.

3. Does not work pretty well with outliers.

4. Non-Homogenous features scales.

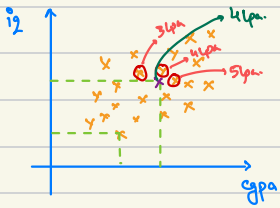
5. Imbalanced Dataset. (Yes → 98% , No → 2%) (Biased Results towards "Yes")

6. Inference and not for prediction. (Acts like Black-Box model)

Advanced KNN

kNN Regressor

cgpa | iq | package.



every point.

1. find out distance blw x_q and all the points in x_train
2. Sort the distances in ascending order & take k values.
3. Calculate mean/median → that's the value of x_q.

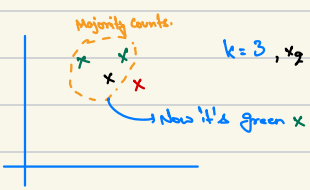
$\mu = \frac{3+4+5}{3} \Rightarrow 4$ lpa.

k-value : High → Underfitting.
low → Overfitting.

Hyperparameters :

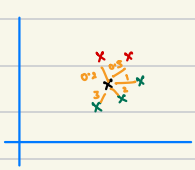
n_neighbors → int : Total value of k.

weights → uniform :



Weight of every neighbor is same.

distance : Now weighted kNN.



k=5, x_q.
uniform kNN → x_q (Green)
Weighted kNN →

$w = \frac{1}{\text{Distance}}$

	dist.	wt.	
red →	0.2	5	5+2 → 7. x_q → red.
red →	0.5	2	
green →	1	1	1+0.5+0.33 → 1.8
green →	2	0.5	
green →	3	0.33	

p : power parameter. → Minkowski

1 → manhatten_distance. (L1)

2 → euclidean_distance (L2) [Default].

Type of Distances :

Euclidean Distance → Shortest dist. btw 2 points. (L2 Norm)

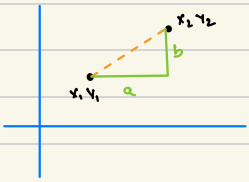
$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

$$d = \sqrt{\sum_{i=1}^{d} (x_{2i} - x_{1i})^2}$$

Problems with L2 : 1. Same scale.

2. Curse of Dimensionality.

Manhattan Distance → Also called taxi-cab distance



$$m = \sum_{i=1}^{d} |x_{2i} - x_{1i}|$$

$$m = |x_2 - x_1| + |y_2 - y_1|$$

What is Minkowski?

$$m = \left(\sum_{i=1}^{d} |x_{2i} - x_{1i}| \right)^{1/1} \longrightarrow p$$

$$d = \left(\sum_{i=1}^{d} (x_{2i} - x_{1i})^2 \right)^{1/2} \longrightarrow p$$

general eqn. $\left(\sum_{i=1}^{d} |x_{2i} - x_{1i}| \right)^{1/p} \longleftarrow p$ $\{p > 0\}$

$p \simeq 0$ Best for High features.

$p \simeq 1$ Best for Low features.

n_jobs : '-1' value divides the workflow for multiple cores of CPU.

algorithm :

Time & Space Complex. :

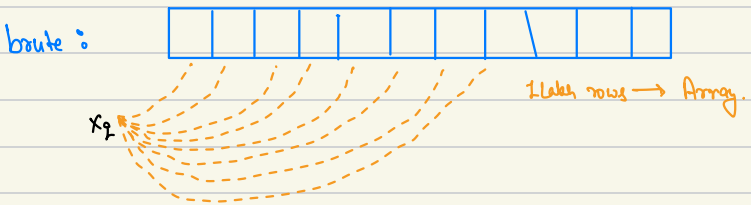
KNN is slow algo. (Lazy learning technique) (All work done during predict phase)

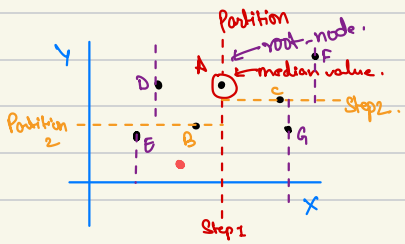
T.C : O(nd) — n → no. of rows in training data. d → no. of features.

S.C : O(nd) — [Stores entire data in RAM].

algorithm : kd_tree

T.C : O(d log n)



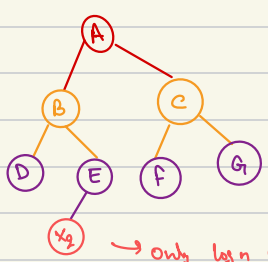


Binary Search Tree is created.

kd_tree

↓

x co-ordinate ⟶ median value



→ only log n comparision.

x_q (5, 10)

algorithm : ball_tree

When n_dims > 15 | 20 (or)